Construcción de Software

Tema 09: Patrones de diseño

Ing. Antonio Arroyo-Paz

Escuela Profesional de Ingeniería de Sistemas · UNSA · Semestre 2026-B

Información de la sesión

SEMANA

09

19/10/2026 – 24/10/2026

Tema 09: Patrones de diseño

Curso	Construcción de Software
Docente	Ing. Antonio Arroyo-Paz
Semestre / Grupo	2026-B · Grupos de Mar y Mié
Guía de laboratorio	N.º 07 — Patrones de diseño
Modalidad	Teoría 100 min · Inicio, Utilidad, Transformación, Práctica, Cierre

Agenda

1. Inicio · 10 min
2. Utilidad · 10 min
3. Transformación · 50 min
4. Práctica grupal · 25 min
5. Cierre · 5 min

Logro de aprendizaje

Al finalizar la sesión, el estudiante **aplica** patrones de diseño GoF para resolver problemas recurrentes de diseño en un sistema real y justifica cuándo no usarlos.

1 Comprende

Explica el propósito, la estructura y las consecuencias de nueve patrones creacionales, estructurales y comportamentales.

2 Aplica

Selecciona y diagrama en grupo el patrón adecuado para escenarios de RutaColca.

3 Evalúa

Evalúa las propuestas de la IA y detecta la sobreaplicación de patrones.

Resultado del estudiante (ABET): C.c — Diseña sistemas con restricciones realistas; C.d — Trabaja en equipo; C.g — Comunica a públicos diversos.

01

SECCIÓN

Inicio

10 min · Saberes previos: ¿qué pasa con este código cuando llega un nuevo método de pago?

¿Qué tiene de malo este código?

- **Recordemos:** PedidoService (Tema 08) y YapeAdapter (Tema 07).
- **Situación:** la cooperativa quiere aceptar Plin y tarjeta, y avisar también por correo.
- **Reto en parejas (3 min):** ¿cuántas partes hay que editar? ¿Qué se rompe si fallamos?

```
class PedidoService {
  void pagar(Pedido p) {
    if (p.metodo().equals("YAPE")) {
      YapeClient y = new YapeClient();
      y.pagar(p.celular(), centimos(p));
    } else if (p.metodo().equals("PLIN")) {
      new PlinApi().transferir(p.celular(), ...);
    } else if (p.metodo().equals("TARJETA")) {
      ...
    }
    whatsapp.enviar(p.productor(), "Pagado");
    stock.descontar(p);
    contabilidad.registrar(p);
  }
}
```

1 2 3

1 · if/else por método 2 · new de clases concretas 3 · avisos acoplados

Figura 1. PedidoService sin patrones: condicionales y dependencias concretas.

02

SECCIÓN

Utilidad

10 min · Qué es un patrón de diseño, por qué usarlo y qué dice la investigación.

¿Qué es un patrón de diseño?

> **Patrón de diseño**
>
> *Describe un problema que se repite en el diseño de software y el núcleo de una solución probada, que puede reutilizarse muchas veces sin aplicarse dos veces de la misma forma.*
>
> — Adaptado de Gamma et al. (1994) y Valente (2020, cap. 6).

Problema recurrente

Solución probada

Consecuencias

Los cuatro elementos de un patrón

- **Vocabulario común:** "usemos un Adaptador" resume una idea de diseño.
- **Experiencia reutilizada:** soluciones probadas por miles de equipos.
- **Diseño flexible:** facilita el cambio sin reescribir.
- **Pero:** no son código para copiar; se adaptan al problema.

Nombre	Adaptador (Adapter)
Problema	El SDK de Yape no tiene la interfaz que espera AgroConecta.
Solución	Una clase que implementa PasarelaPago y traduce las llamadas al SDK.
Consecuencias	+ Cambiar de proveedor sin tocar PedidoService. − Una clase más que mantener.

Figura 2. Elementos de un patrón según Gamma et al. (1994), con el ejemplo del Adaptador.

Estado del Arte: patrones de diseño e IA

Autor (año)	Propuesta	Aporte principal	Relación con el tema
Prechelt et al. (2001)	Experimento de mantenimiento con y sin patrones	Los patrones ayudan en ciertos cambios; la flexibilidad innecesaria puede estorbar	Usar un patrón solo cuando el problema lo pide
Zhang y Budgen (2012)	Revisión sistemática sobre la eficacia de los patrones GoF	La evidencia empírica sobre sus beneficios es limitada y mixta	Justificar cada patrón con el problema concreto
Pandey et al. (2025)	Reconocimiento de patrones GoF en código Java con modelos de lenguaje	RoBERTa logró un F1 medio de 0.91 en el repositorio P-MARt	La IA puede reconocer patrones en código existente
Pan et al. (2025)	Sesgos de los LLM de código al reconocer y generar patrones	Los sesgos afectan la fiabilidad de las tareas posteriores	Verificar siempre lo que la IA propone
Chowdhury et al. (2026)	Detección de patrones con LLM desde código, PlantUML o texto	Singleton y Decorator, los más fáciles; Composite, el más difícil	El formato de entrada no cambió el resultado

Búsqueda en IEEE Xplore, Springer, ACM y arXiv (2001–2026).

03

SECCIÓN

Transformación

50 min · Tema 1: catálogo GoF y principios. Tema 2: creacionales. Tema 3: estructurales. Tema 4: comportamentales.

El catálogo GoF: 23 patrones en 3 familias

Familia	Qué resuelve	Patrones GoF (★ = se ven hoy)
Creacionales (5)	Cómo crear objetos sin depender de sus clases concretas	Factory Method ★, Singleton ★, Abstract Factory, Builder, Prototype
Estructurales (7)	Cómo componer clases y objetos en estructuras mayores	Adapter ★, Facade ★, Decorator ★, Proxy ★, Bridge, Composite, Flyweight
Comportamentales (11)	Cómo repartir responsabilidades y comunicar objetos	Strategy ★, Observer ★, Template Method ★, Visitor, Iterator, Command, State, Chain of Responsibility, Mediator, Memento, Interpreter

Principios detrás de los patrones

Encapsular lo que varía	Programar para interfaces	Composición > herencia	Abierto/cerrado	Bajo acoplamiento
1	2	3	4	5
Separar lo que cambia (métodos de pago) de lo estable (el pedido).	PedidoService depende de PasarelaPago, no de YapeAdapter.	Combinar objetos en ejecución en vez de crear subclases.	Un método de pago nuevo es una clase nueva, sin editar las demás.	Cada clase conoce lo mínimo de las otras y se prueba sola.

Fuente: Gamma et al. (1994); Valente (2020), cap. 5 "Princípios de projeto". Estos principios explican por qué funcionan los patrones.

Herencia vs. composición

Herencia ("es un")

Características
la subclase extiende a la superclase al compilar.

Ventajas
simple; reutiliza código y usa polimorfismo.

Limitaciones
acoplamiento fuerte; explosión de subclases (YapeConLog…).

Cuándo usarlo
relación estable, como en Template Method.

vs

Composición ("tiene un")

Características
un objeto delega en otro mediante una interfaz.

Ventajas
combina comportamientos en ejecución; menos acoplamiento.

Limitaciones
más objetos y más indirección.

Cuándo usarlo
comportamientos que varían: Strategy, Decorator.

Creacional · Fábrica (Factory)

- **Problema:** los new de cada adaptador están repartidos en el código.
- **Solución:** un método decide qué clase concreta crear.
- **En AgroConecta:** PasarelaFactory crea Yape, Plin o Tarjeta.
- **Úselo cuando:** varias clases implementen la misma interfaz.

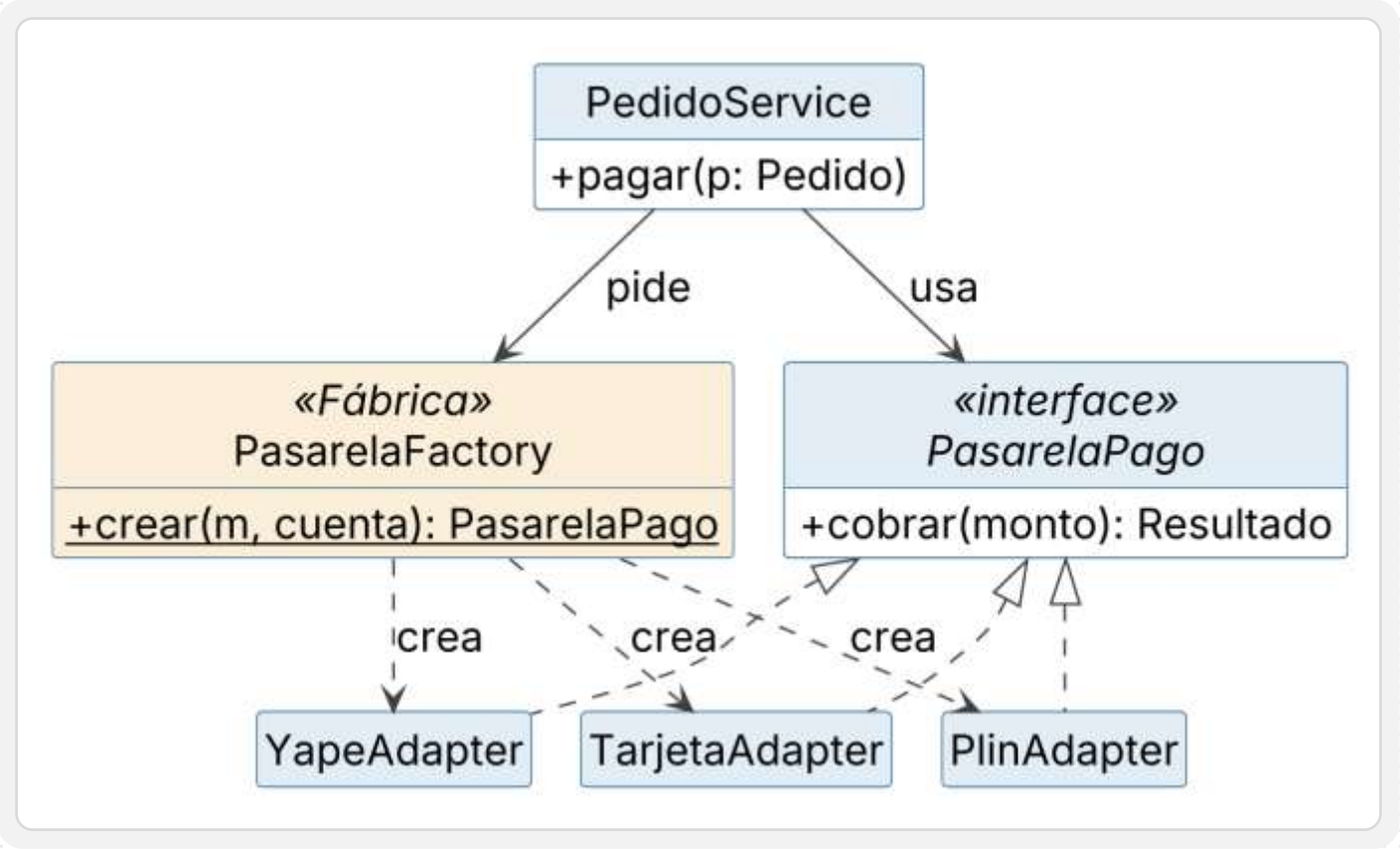


Figura 3. Fábrica de pasarelas de pago de AgroConecta (PlantUML).

Fábrica en Java

PasarelaFactory.java

```
enum MetodoPago { YAPE, PLIN, TARJETA }

class PasarelaFactory {
    static PasarelaPago crear(MetodoPago metodo, String cuenta) {
        return switch (metodo) {
            case YAPE   -> new YapeAdapter(new YapeClient(), cuenta);
            case PLIN   -> new PlinAdapter(new PlinApi(), cuenta);
            case TARJETA -> new TarjetaAdapter(cuenta);
        };
    }
}

// En PedidoService ya no hay new de adaptadores
PasarelaPago pasarela =
        PasarelaFactory.crear(pedido.metodo(), pedido.cuenta());
Resultado r = pasarela.cobrar(pedido.total());
```

Línea 1 · Variantes

Un enum con los métodos de pago aceptados.

Líneas 3–11 · Fábrica

Un solo lugar decide qué clase concreta crear.

Líneas 14–16 · Cliente

Depende solo de la interfaz PasarelaPago.

Creacional · Singleton

- **Problema:** se necesita una sola instancia compartida.
- **Solución:** constructor privado y un método estático get().
- **En AgroConecta:** Configuracion (moneda, comisiones).
- **Cuidado:** es estado global y dificulta las pruebas; en Spring los beans ya son únicos.



Figura 4. Singleton para la configuración de AgroConecta (PlantUML).

Estructural · Adaptador (Adapter)

- **Problema:** el SDK de Yape no coincide con PasarelaPago.
- **Solución:** una clase que implementa la interfaz esperada y traduce las llamadas.
- **En AgroConecta:** YapeAdapter (Tema 07).
- **Úselo cuando:** integre APIs externas o código heredado.

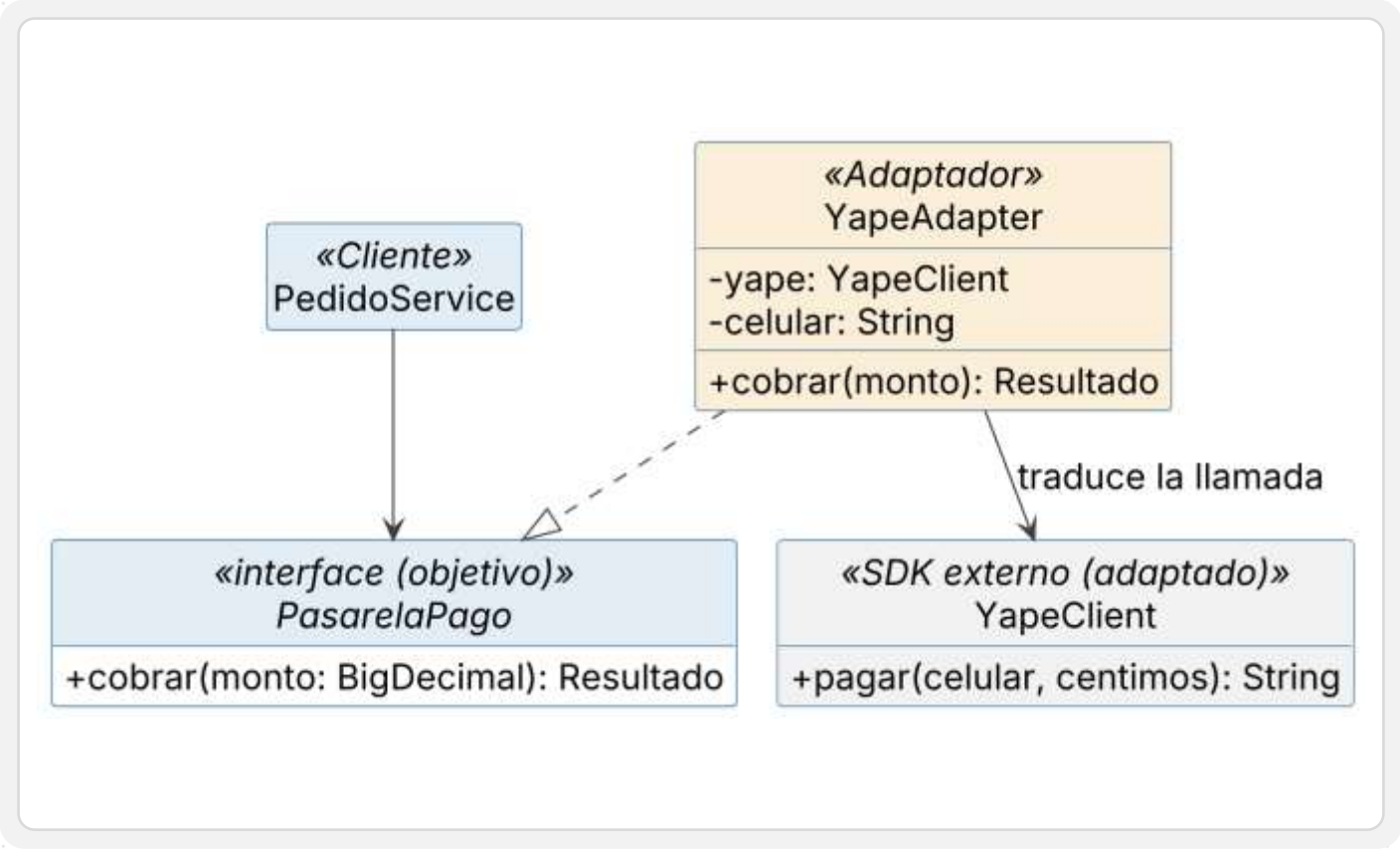


Figura 5. YapeAdapter adapta el SDK de Yape a PasarelaPago (PlantUML).

Adaptador en Java

YapeAdapter.java

```java
// SDK de Yape: otra interfaz, no se puede modificar
class YapeClient {
    String pagar(String celular, long centimos) { ... }
}

class YapeAdapter implements PasarelaPago {
    private final YapeClient yape;
    private final String celular;

    YapeAdapter(YapeClient yape, String celular) {
        this.yape = yape;
        this.celular = celular;
    }

    @Override
    public Resultado cobrar(BigDecimal monto) {
        long centimos = monto.movePointRight(2).longValueExact();
        String operacion = yape.pagar(celular, centimos);
        return new Resultado(operacion != null, operacion);
    }
}
```

Líneas 2–4 · Adaptado

SDK externo con otra firma: celular y céntimos.

Línea 6 · Adaptador

Implementa la interfaz que espera AgroConecta.

Líneas 16–20 · Traducción

Convierte el monto y el resultado entre ambos mundos.

Estructural · Fachada (Facade)

- **Problema:** confirmar un pedido exige coordinar cuatro clases.
- **Solución:** una clase con una interfaz simple oculta el subsistema.
- **En AgroConecta:** CheckoutFacade.confirmarYPagar(pedido).
- **Úselo cuando:** el cliente no deba conocer los detalles internos.

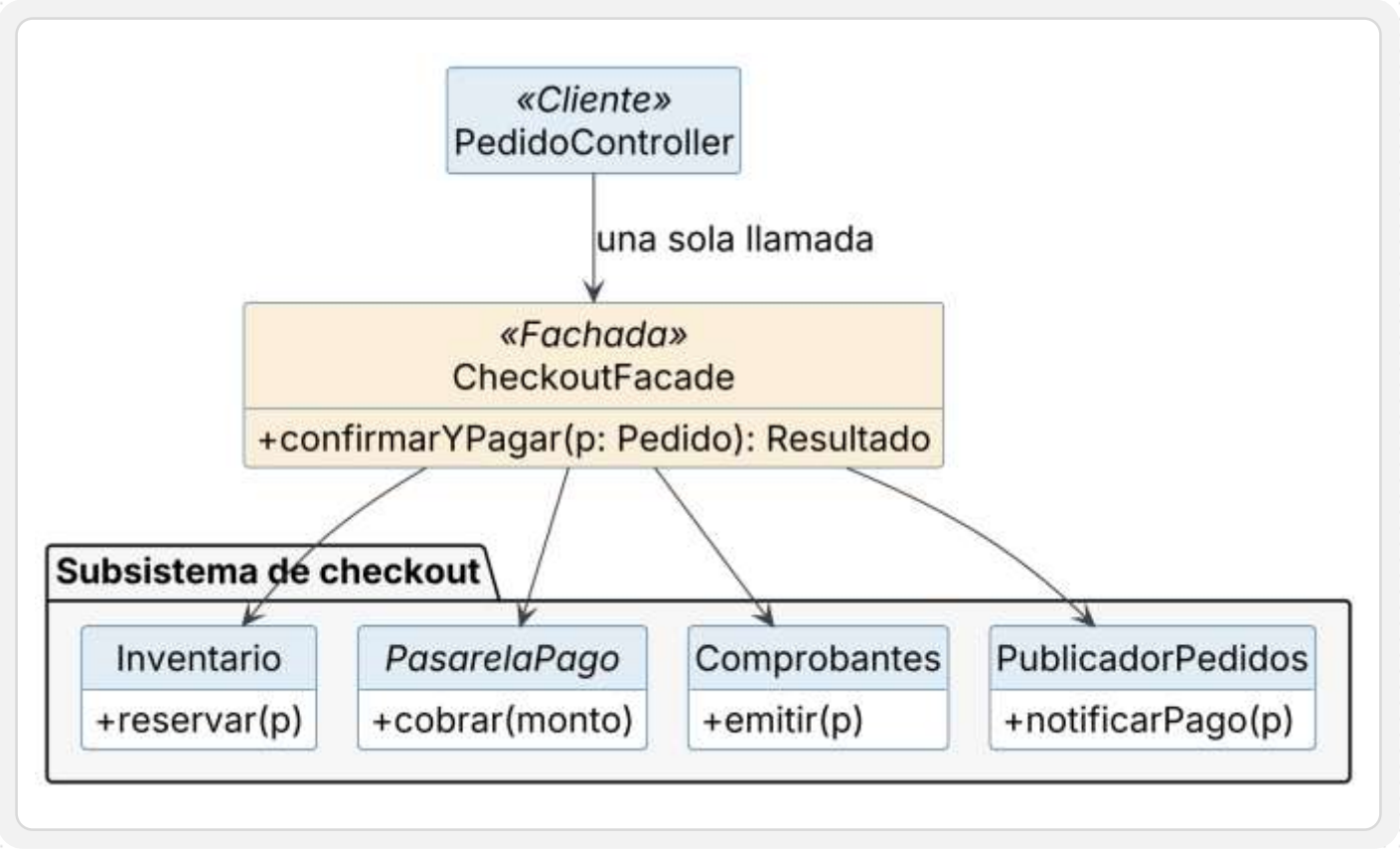


Figura 6. CheckoutFacade simplifica el subsistema de checkout (PlantUML).

Estructural · Decorador (Decorator)

- **Problema:** agregar registro y reintentos sin una subclase por combinación.
- **Solución:** objetos que envuelven a otro con la misma interfaz.
- **En AgroConecta:** PasarelaConRegistro y PasarelaConReintentos.
- **Úselo cuando:** las funciones opcionales se combinen.

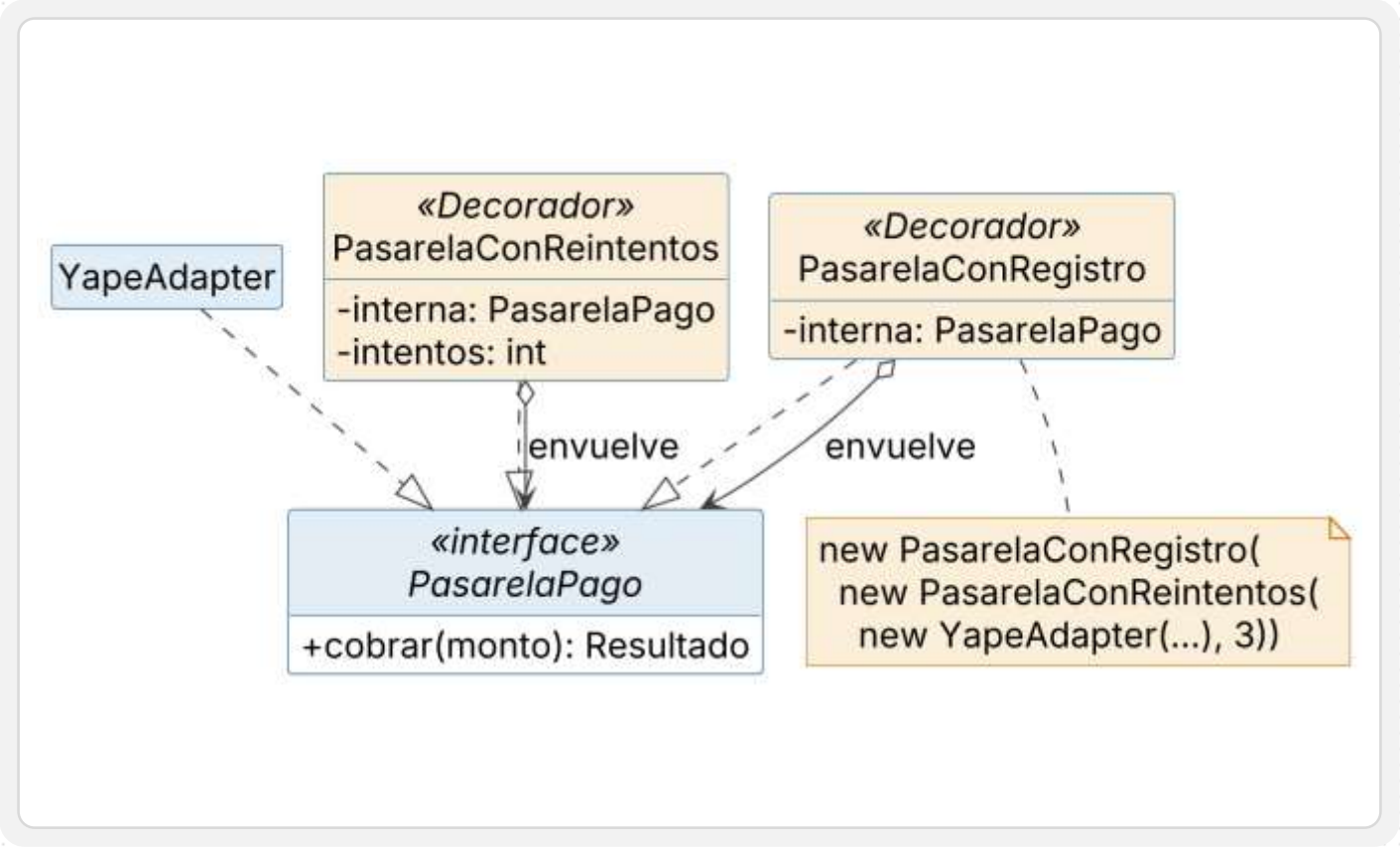


Figura 7. Decoradores de PasarelaPago (PlantUML).

Decorador en Java

PasarelaConReintentos.java

```
class PasarelaConReintentos implements PasarelaPago {
    private final PasarelaPago interna;
    private final int intentos;

    PasarelaConReintentos(PasarelaPago interna, int intentos) {
        this.interna = interna;
        this.intentos = intentos;
    }

    @Override
    public Resultado cobrar(BigDecimal monto) {
        for (int i = 1; i < intentos; i++) {
            try { return interna.cobrar(monto); }
            catch (RuntimeException e) { /* falla temporal */ }
        }
        return interna.cobrar(monto);    // último intento
    }
}
// Se apilan en ejecución:
// new PasarelaConRegistro(new PasarelaConReintentos(yape, 3))
```

Línea 1 · Misma interfaz

El decorador también es una PasarelaPago.

Línea 2 · Envuelve

Guarda la pasarela a la que agrega comportamiento.

Líneas 19–20 · Composición

Registro + reintentos sin crear subclases.

Estructural · Proxy

- **Problema:** cada búsqueda en el catálogo consulta la base de datos.
- **Solución:** un intermediario con la misma interfaz controla el acceso.
- **En AgroConecta:** CatalogoProxy guarda los resultados en caché.
- **Variantes:** caché, control de acceso, carga diferida, remoto.

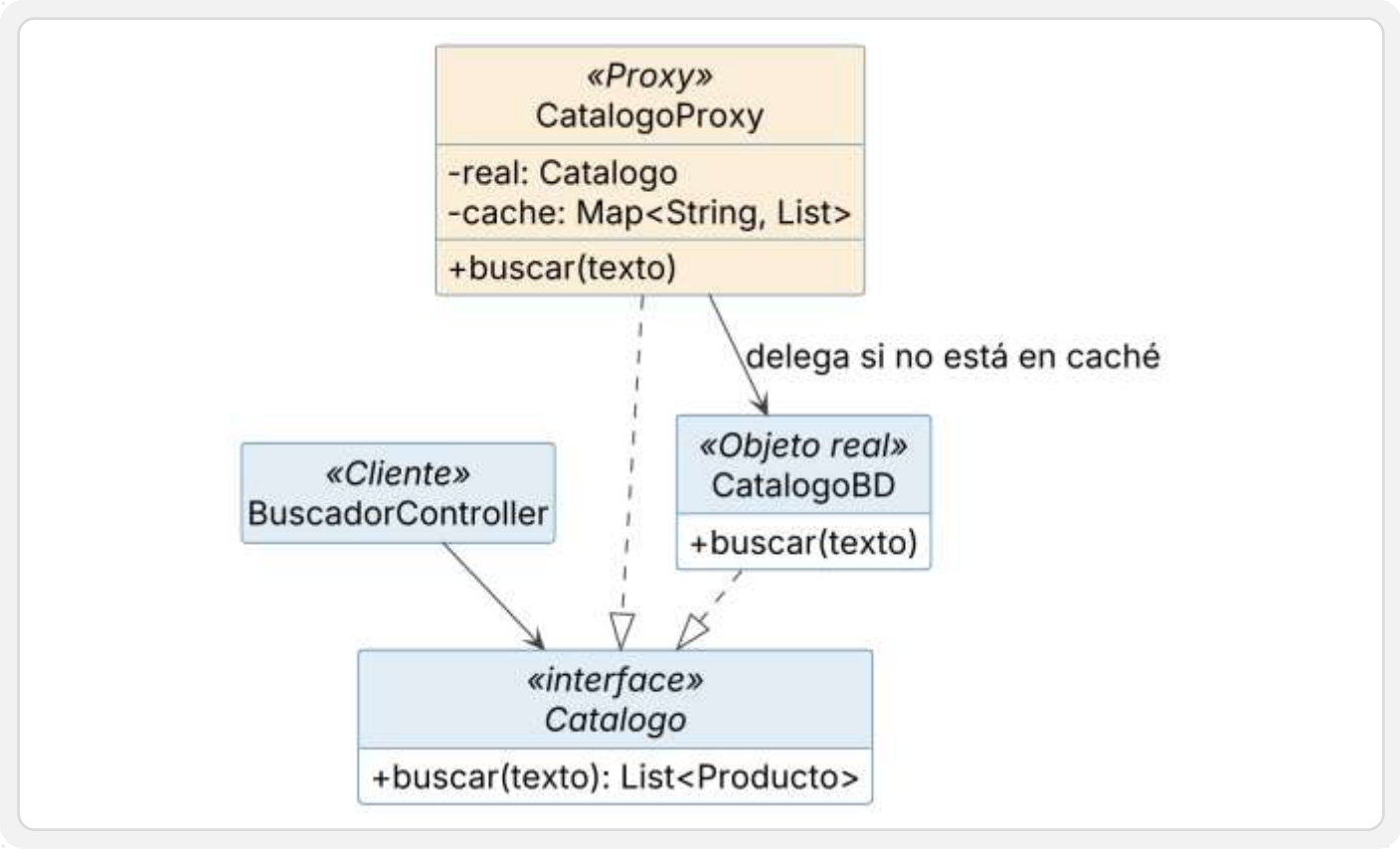


Figura 8. CatalogoProxy con caché delante de la base de datos (PlantUML).

Comportamental · Strategy

- **Problema:** el costo de envío cambia según la modalidad.
- **Solución:** una interfaz por familia de algoritmos; el contexto delega.
- **En AgroConecta:** TarifaEnvio: recojo, Majes y Arequipa.
- **Úselo cuando:** el algoritmo se elija en ejecución.

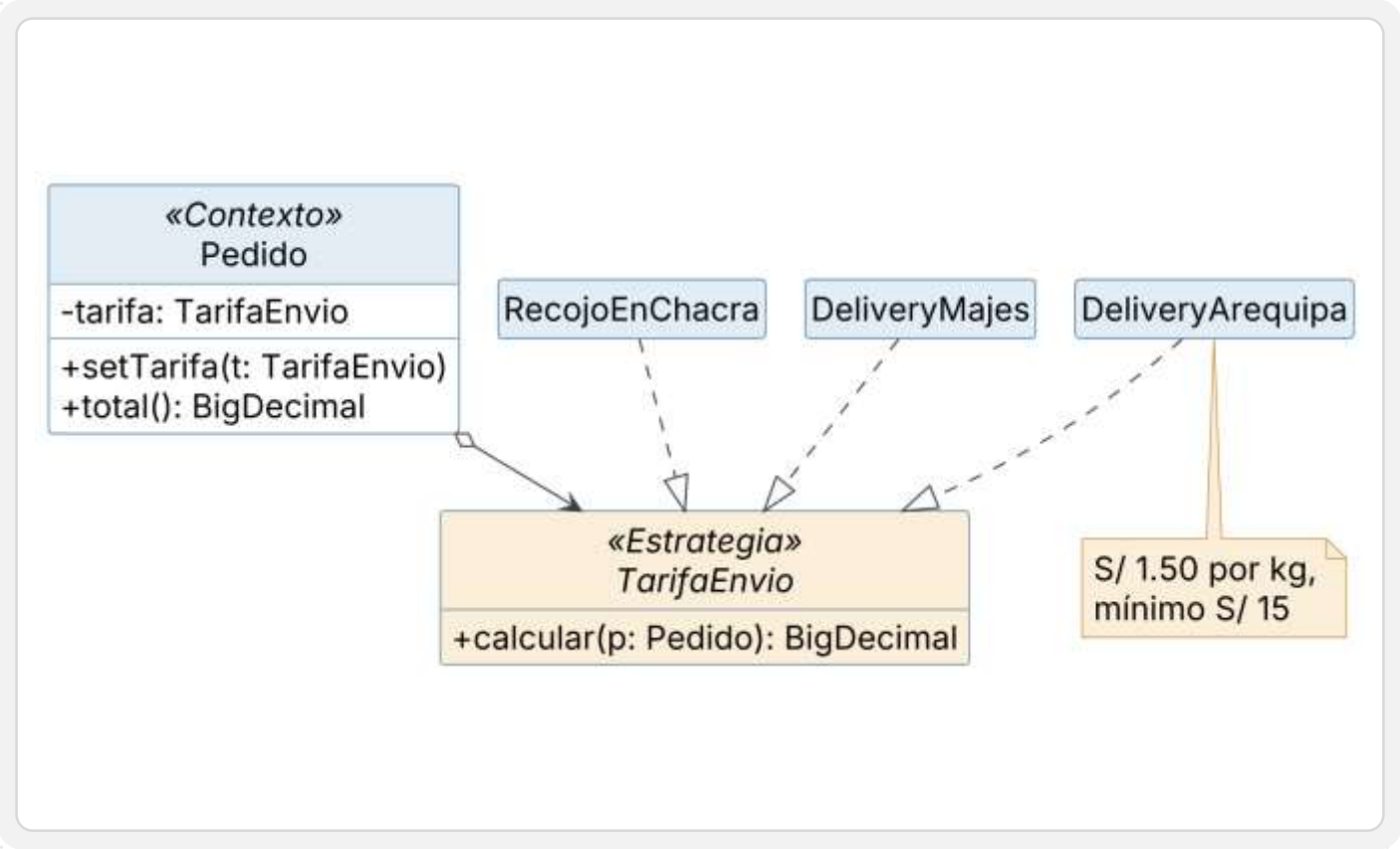


Figura 9. Estrategias de tarifa de envío de un pedido (PlantUML).

Strategy en Java

TarifaEnvio.java

```
interface TarifaEnvio {
    BigDecimal calcular(Pedido pedido);
}

class DeliveryArequipa implements TarifaEnvio {
    public BigDecimal calcular(Pedido p) {
        BigDecimal porKg = new BigDecimal("1.50");
        BigDecimal minimo = new BigDecimal("15.00");
        return p.pesoKg().multiply(porKg).max(minimo);
    }
}

class Pedido {
    private TarifaEnvio tarifa = new RecojoEnChacra();

    void setTarifa(TarifaEnvio tarifa) { this.tarifa = tarifa; }

    BigDecimal total() {
        return subtotal.add(tarifa.calcular(this));
    }
}
```

Líneas 1–3 · Estrategia

Una interfaz para la familia de algoritmos.

Líneas 5–11 · Concreta

S/ 1.50 por kg, con un mínimo de S/ 15.

Líneas 13–21 · Contexto

Pedido delega el cálculo; la tarifa se cambia en ejecución.

Comportamental · Observador (Observer)

- **Problema:** al pagar hay que avisar, descontar stock y registrar.
- **Solución:** el sujeto notifica a una lista de observadores.
- **En AgroConecta:** PublicadorPedidos y sus suscriptores.
- **Úselo cuando:** varios objetos reaccionen a un evento.

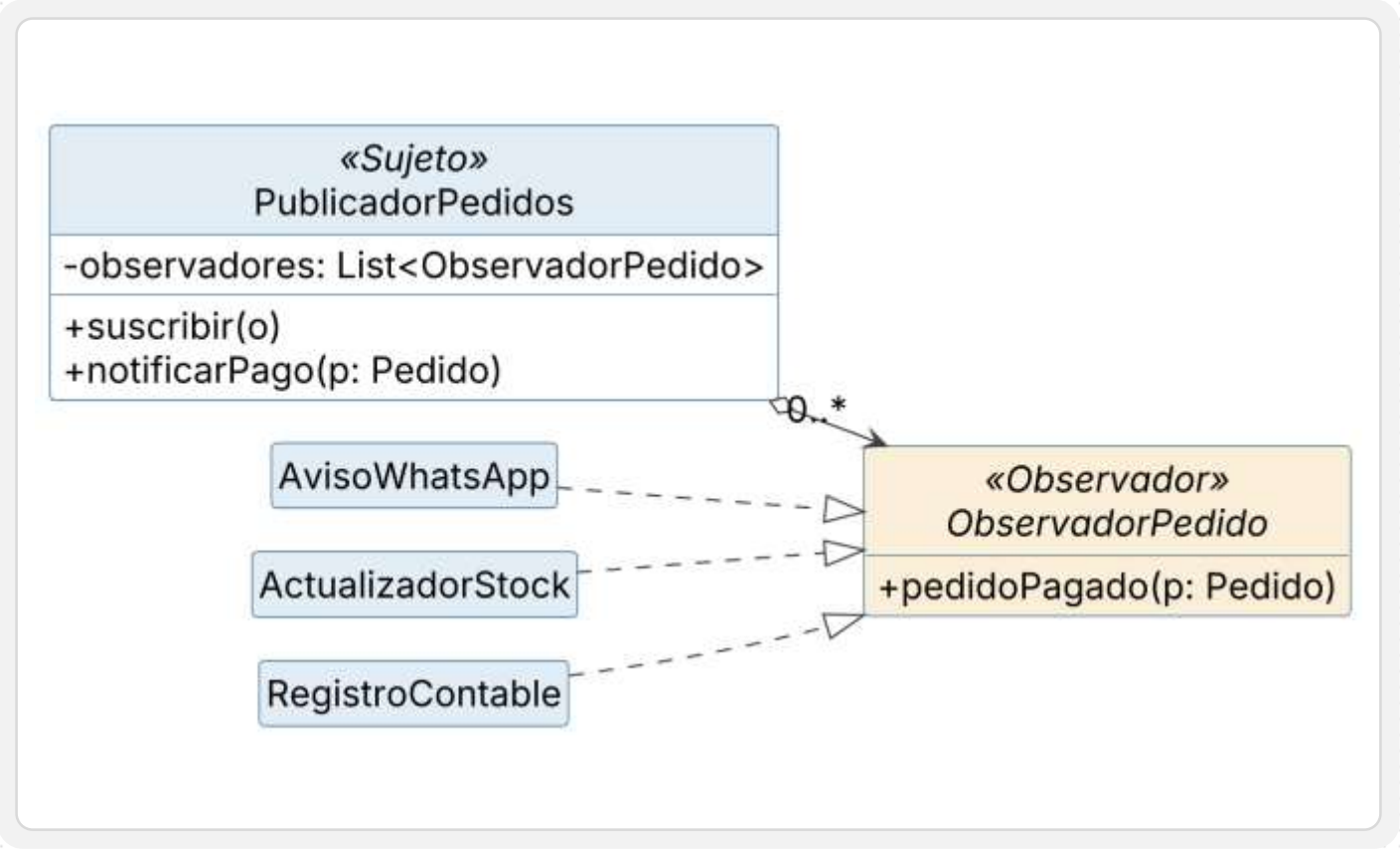


Figura 10. Observadores del evento "pedido pagado" (PlantUML).

Observador en Java

PublicadorPedidos.java

```
interface ObservadorPedido {
    void pedidoPagado(Pedido pedido);
}

class PublicadorPedidos {
    private final List<ObservadorPedido> observadores =
            new ArrayList<>();

    void suscribir(ObservadorPedido o) { observadores.add(o); }

    void notificarPago(Pedido pedido) {
        for (ObservadorPedido o : observadores) {
            o.pedidoPagado(pedido);
        }
    }
}

// Configuración, una sola vez al iniciar la aplicación
eventos.suscribir(new AvisoWhatsApp());
eventos.suscribir(new ActualizadorStock());
```

Líneas 1–3 · Observador

Lo que cada interesado debe implementar.

Líneas 5–16 · Sujeto

Avisa a todos sin conocer sus clases concretas.

Líneas 19–20 · Extensión

Un nuevo interesado es una clase nueva; el pago no cambia.

Comportamental · Template Method

- **Problema:** los reportes repiten el mismo algoritmo con pasos distintos.
- **Solución:** la superclase fija el algoritmo; las subclases completan los pasos.
- **En AgroConecta:** Reporte → ReporteVentasCsv y ReporteVentasPdf.
- **Úselo cuando:** el esqueleto sea estable.

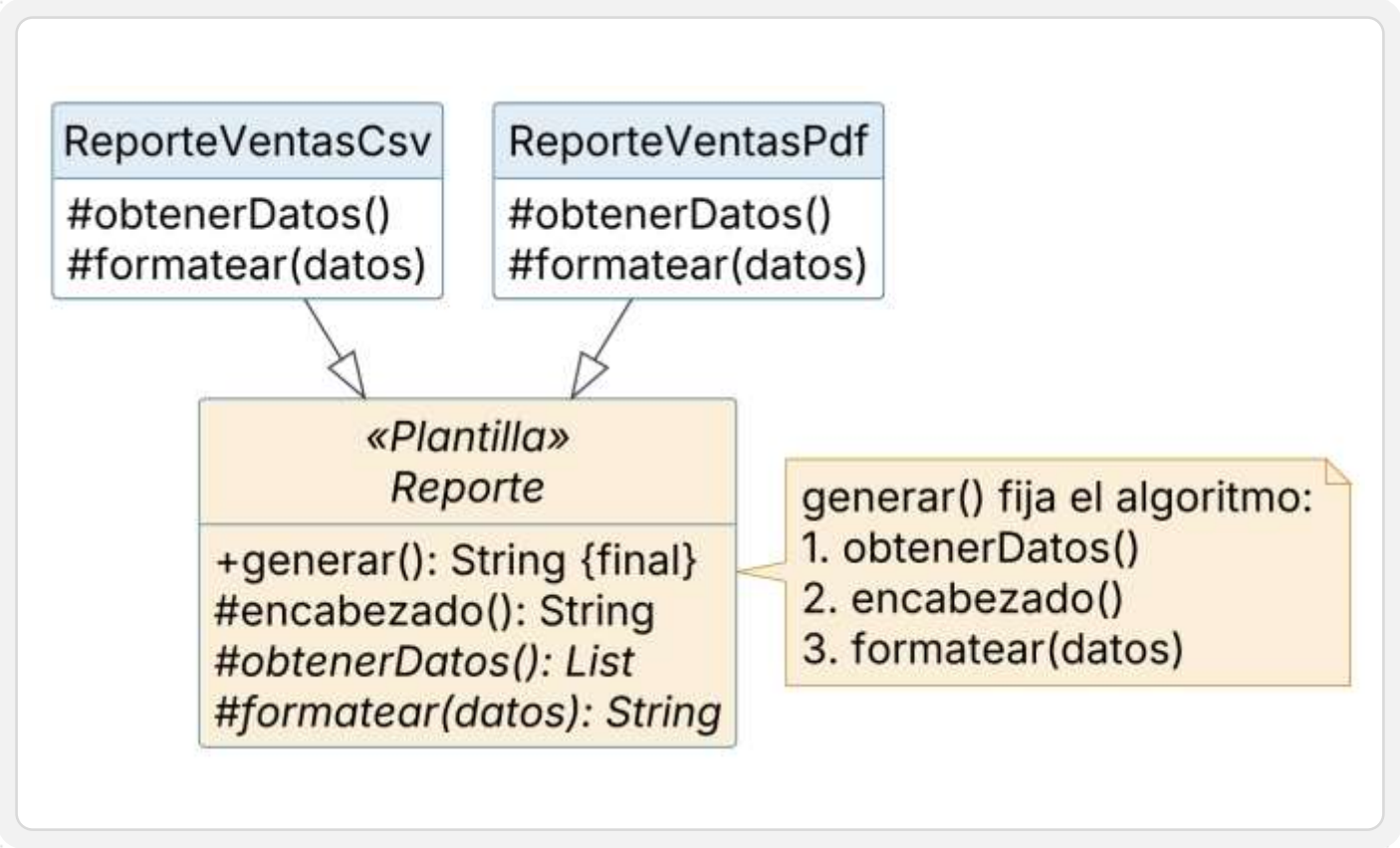


Figura 11. Plantilla de generación de reportes (PlantUML).

Los nueve patrones en una tabla

Patrón	Problema	Solución	En AgroConecta
Fábrica	new repartidos por el código	Un método crea la clase concreta	PasarelaFactory
Singleton	Una sola instancia compartida	Constructor privado + get()	Configuracion
Adaptador	Interfaces incompatibles	Clase que traduce llamadas	YapeAdapter
Fachada	Subsistema complejo	Interfaz simple de entrada	CheckoutFacade
Decorador	Funciones opcionales combinables	Envolver con la misma interfaz	PasarelaConReintentos
Proxy	Acceso costoso o controlado	Intermediario con la misma interfaz	CatalogoProxy
Strategy	Algoritmos intercambiables	Interfaz + delegación	TarifaEnvio
Observador	Varios reaccionan a un evento	Lista de suscriptores	PublicadorPedidos
Template Method	Algoritmo con pasos variables	Esqueleto en la superclase	Reporte

Todas las clases de los ejemplos se compilaron y ejecutaron con Java 21.

Cuándo NO usar patrones: la "paternite"

Paternite

El mayor riesgo de los patrones de diseño es su sobreaplicación: no fuerce un problema a encajar en un patrón cuando una solución tradicional funciona mejor.

— Ousterhout (2018), citado por Valente (2020), cap. 6 (traducción propia).

¿Hay variantes reales?

¿Basta un new?

Simple primero

Pedirle un patrón a la IA (y verificarlo)

prompt-patron.txt

```
Actúa como diseñador de software experto en patrones GoF.
Contexto: [pegar PedidoService.java y la historia HU-07].
Problema: cada método de pago nuevo obliga a editar
un if/else en PedidoService.

Tarea:
1. Identifica el problema de diseño.
2. Propón UN patrón y justifica con un principio.
3. Dibuja el diagrama de clases en PlantUML.
4. Explica qué pasa si NO usamos el patrón.

Reglas: Java 21; sin dependencias nuevas.
Si el problema no justifica un patrón, dilo.
```

Línea 2 · Contexto real

Código e historia de usuario, no una descripción vaga.

Líneas 10 y 13 · Sin paternite

Pedir la opción "sin patrón" evita la sobreaplicación.

Después · Verificar

Compilar, probar y registrar en la bitácora de IA.

04

SECCIÓN

Práctica

25 min · Actividad grupal: patrones de diseño para RutaColca.

Actividad grupal: patrones para RutaColca

25 min
Duración de la actividad

3–4
Integrantes por grupo

5
Escenarios de diseño

Descripción del caso

RutaColca (caso AG-08) crece: acepta tarjeta, Yape y PayPal; el precio varía por temporada, grupo o estudiante; al confirmar una reserva se avisa al turista, al guía y a la agencia; la API de AUTOCOLCA no coincide con la interfaz esperada, y la disponibilidad se consulta cientos de veces.

RETO

¿En cuál de los cinco escenarios NO usarían un patrón? Justifíquenlo con el criterio de la "paternite".

Práctica grupal en clase

ACTIVIDAD GRUPAL

AG-09

Patrones de diseño en RutaColca

Semana 09 · 19–24/10/2026
Duración: 25 min
Modalidad: grupos de 3 a 4

1

Identificar · 7 min
Para cada escenario: problema de diseño y patrón candidato, o ninguno.

2

Diseñar · 10 min
Diagrama de clases en PlantUML del patrón para dos escenarios.

3

Verificar · 5 min
Pedir a la IA una alternativa y contrastarla con su diseño.

4

Socializar · 3 min
Un grupo por patrón explica su decisión y lo que rechazó de la IA.

Rúbrica de la actividad AG-09 (20 puntos)

Criterio	Logrado (4 pts)	En proceso (2 pts)
Identificación	Problema de diseño claro en los cinco escenarios	Problema confuso o incompleto
Elección del patrón	Patrón adecuado y justificado con un principio	Patrón sin justificación
Diagrama de clases	Roles visibles: interfaz, clases concretas y cliente	Diagrama incompleto o sin roles
Uso crítico de la IA	Propuesta de la IA contrastada y registrada	Se copia la respuesta de la IA
Paternite	Detecta dónde no conviene un patrón	Aplica patrones en todos los casos

05

SECCIÓN

Cierre

5 min · Ticket de salida, conclusiones y próxima sesión.

Ticket de salida

Responda en una línea (2 min)

Un banco nuevo ofrece pagos con su propia API. ¿Qué patrón usaría en AgroConecta y qué clase nueva crearía?

— Respuesta individual en el Aula Virtual.

Adaptador | **Fábrica** | **Abierto/cerrado**

Conclusiones

01

Soluciones probadas

Un patrón nombra un problema recurrente y su solución; da vocabulario común al equipo.

02

Composición

Fábrica, Adaptador, Decorador, Strategy y Observador se apoyan en programar para interfaces.

03

Sin paternite

Use un patrón solo si el problema lo pide. La IA propone; el equipo verifica.

Próxima sesión: Tema 10 — Diseño de la arquitectura · Guía de laboratorio N.º 07 durante la semana

Referencias

Freeman, E., & Robson, E. (2020). Head first design patterns (2nd ed.). O'Reilly Media.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.

Ousterhout, J. (2018). A philosophy of software design. Yaknyam Press.

Refactoring.Guru. (s. f.). Patrones de diseño. https://refactoring.guru/es/design-patterns

Valente, M. T. (2020). Engenharia de software moderna (cap. 5: Princípios de projeto; cap. 6: Padrões de projeto). Independente. https://engsoftmoderna.info/

Referencias

Chowdhury, O., Savarimuthu, B. T. R., & Licorish, S. A. (2026). A pilot study on detecting software design patterns with large language models: An empirical evaluation. En ENASE 2026. https://arxiv.org/abs/2604.17329

Pan, Z., Song, X., Wang, Y., Cao, R., Li, B., Li, Y., & Liu, H. (2025). Do code LLMs understand design patterns? En LLM4Code Workshop, ICSE 2025. https://arxiv.org/abs/2501.04835

Pandey, S. K., Chand, S., Horkoff, J., Staron, M., Ochodek, M., & Durisic, D. (2025). Design pattern recognition: A study of large language models. Empirical Software Engineering, 30(3), 69. https://doi.org/10.1007/s10664-025-10625-1

Prechelt, L., Unger, B., Tichy, W. F., Brössler, P., & Votta, L. G. (2001). A controlled experiment in maintenance comparing design patterns to simpler solutions. IEEE Transactions on Software Engineering, 27(12), 1134–1144.

Zhang, C., & Budgen, D. (2012). What do we know about the effectiveness of software design patterns? IEEE Transactions on Software Engineering, 38(5), 1213–1231.

¿Preguntas?

Gracias por su atención



Ing. Antonio Arroyo-Paz
aarroyop@unsa.edu.pe